

基于无人机俯视视频的 城市道路拥堵实时分析系统

(作者姓名) 1

(1 单位名称, 城市 邮编)

摘 要

随着城市化进程加快和机动车保有量持续增长, 城市道路交通拥堵已成为制约交通效率与出行品质的重要问题。传统拥堵研判多依赖路侧固定检测器、浮动车数据或宏观交通指数, 存在布设成本高、空间覆盖有限、难以快速机动部署等不足。本文面向无人机垂直俯视场景, 设计并实现一套“车辆检测—轨迹跟踪—拥堵量化—Web实时展示”一体化分析系统。感知层采用VisDrone预训练模型域适应与X-AnyLabeling自标注数据集微调相结合的两阶段YOLOv12训练策略, 配合ByteTrack实现多目标跟踪; 算法层在二维像素空间提取平均速度、车队内车距中位数及车距时序波动等特征, 构建含车队切分、连续状态机、Hold挂起与Dist Warmup抗抖动机制的拥堵指数模型, 输出0—100分拥堵指数Ct; 应用层基于FastAPI实现MJPEG视频流、WebSocket指标推送与90秒滚动曲线展示。实验表明, 系统可在消费级GPU上稳定运行, 能够区分畅通、有序等待、走走停停及路口等灯等典型场景, 为无人机临时交通监测提供可行方案。

关键词: 无人机; 道路交通拥堵; YOLOv12; 拥堵指数; 实时分析; VisDrone

Abstract: With accelerating urbanization, urban road congestion has become a major constraint on travel efficiency. This paper presents an integrated system for real-time congestion analysis from UAV vertical top-down video. A two-stage YOLOv12 pipeline—VisDrone domain adaptation and fine-tuning on a self-annotated X-AnyLabeling dataset—is combined with ByteTrack. Congestion features are extracted in pixel space, and a CongestionCalculator with platoon splitting, continuous state machine, and anti-jitter mechanisms outputs a 0-100 index. A FastAPI platform provides MJPEG streaming, WebSocket metrics, and rolling charts. Results show stable real-time operation across typical traffic scenarios.

Keywords: UAV; road congestion; YOLOv12; congestion index; real-time analysis

0 引 言

随着经济社会发展和城市化进程加快, 交通拥堵问题日益突出。交通拥堵不仅降低出行效率、增加能源消耗和环境污染, 也提高了城市交通治理成本。准确研判道路运行状态、及时识

别拥堵态势，对交通规划、信号控制与出行诱导具有重要意义。

现有拥堵研究与方法大致可分为三类。其一，基于统计与指数建模的方法。张溪等[1]利用交通运行指数历史数据，采用时间序列法对城市道路交通拥堵进行研判与预测，为管理部门制定措施和出行者规避高峰提供了数据支撑。其二，基于机器学习与智能算法的预测及评价方法。时柏营等[2]提出SSA-SVM短时交通拥堵指数预测模型；朱云等[3]基于模糊层次分析与神经网络建立城市道路交通拥堵评价模型；盖健[4]和潘旭[5]则从大数据、物联网与多源数据融合角度，讨论拥堵态势感知、演化机理及智能疏导决策。其三，基于视觉识别的方法。罗运鹏等[6]对比多种深度学习检测算法，指出YOLO-v8在道路拥堵识别中兼顾速度与精度，适合实时场景。黄晓虹等[7]从城市道路交通供需演变与综合治理角度，强调建管并重背景下的精细化治理需求。

然而，上述研究多面向固定路侧数据、宏观指数预测或城市级治理框架，对无人机俯视视频这一机动、灵活、可快速部署的数据源关注不足。UAV在约80m高度垂直俯拍时，可获取大范围路网图像，但存在目标尺度小、检测抖动大、路口多队列并存等特点，直接套用地面视角模型或简单一维车距统计易产生误判。

针对上述问题，本文设计并实现一套面向UAV俯视视频的道路拥堵实时分析系统，主要工作如下：（1）采用X-AnyLabeling对项目场景数据进行标注，基于VisDrone预训练权重完成域适应与二次微调，构建适用于俯视车辆检测的YOLOv12模型；（2）提出面向像素空间的CongestionCalculator拥堵指数算法，包含车队切分、连续B/C区映射、路口混行降分及多项抗抖动策略；（3）实现FastAPI与WebSocket实时分析平台，支持参数热更新、逐帧调试与指标CSV导出。

1 系统总体架构

本文系统采用“感知层—数据层—算法层—应用层”四层架构，实现从视频输入到拥堵指数输出的完整链路，总体数据流如下：UAV俯视视频输入后，经YOLOv12检测与ByteTrack跟踪得到带ID的车辆轨迹；DataCleaner对轨迹进行平滑与窗口管理；CongestionCalculator计算Ct及场景标签；最终由Web平台实时展示并支持导出。

感知层负责车辆检测与多目标跟踪。输入为UAV垂直俯视视频流，经YOLOv12模型输出检测框，再由ByteTrack关联得到稳定track_id与帧间位移。推理时采用隔帧策略：每2帧执行一次完整检测，中间帧利用上一帧速度做匀速外推，在保证轨迹连续性的同时降低算力占用。模型优先加载ONNX权重，通过DirectML在AMD GPU上加速。

数据层对原始检测进行清洗，包括EMA坐标平滑、死区过滤与3s滑动窗口维护，以抑制悬停微抖和框缘抖动。其中EMA系数 α 默认0.4，在响应速度与平滑程度之间折中；死区阈值默认2px，用于消除无人机轻微悬停造成的伪位移；滑动窗口长度与算法层一致，按30FPS折算为90帧。

算法层基于清洗后的轨迹计算平均速度 V_{avg} 、帧车距 D_t 、车距波动 σ_{dist} 及静止占比 r_{stop} ，经CongestionCalculator输出拥堵指数 C_t 与场景标签。指数取值0—100， $C_t < 60$ 判定畅通， $C_t \geq 60$ 判定拥堵。场景标签包括畅通、有序等待、走走停停、慢速跟车、路口等灯等，便于用户理解当前交通状态。

应用层通过FastAPI提供Web服务，向前端推送MJPEG视频流与WebSocket指标，并支持90s滚动曲线、调试面板及CSV导出。前端布局采用"左侧控制—右侧视频与图表"模式，视频区仅叠加检测框、不遮挡画面，所有数值指标集中于页面面板展示。

与盖健[4]、潘旭[5]强调的多源大数据平台不同，本文系统聚焦单路UAV视频流的轻量级实时分析，更适合临时监测、应急勘查和算法验证场景。系统各模块解耦设计：检测模型、拥堵算法与Web服务可独立升级，便于后续扩展多路视频或与其他交通数据融合。

2 车辆检测模型构建

2.1 数据集标注

本文使用X-AnyLabeling对自采集UAV俯视交通视频帧进行矩形框标注，标注类别为道路车辆。与VisDrone公开数据集相比，自建数据更贴近本项目实际采集条件，包括固定俯视高度、特定道路类型及光照环境，有利于降低域偏移，提高在实际部署视频上的检测召回率。

2.2 两阶段迁移学习

参考罗运鹏等[6]关于YOLO系列在道路识别中实时性与准确性的结论，本文采用YOLOv12作为基础检测框架，训练分为两阶段。

阶段一为VisDrone域适应。以VisDrone预训练YOLOv12权重为基座，在VisDrone合并数据集上训练，输入分辨率1280，训练50 epoch，batch size为4，目的是获得UAV俯视小目标检测的基础能力。

阶段二为自标注数据微调。以阶段一最优权重为基座，在X-AnyLabeling标注后合并的merged_vehicle数据集上继续训练，参数为1280分辨率、40 epoch、batch size 4，目的是适配本项目实际场景中的车型、道路宽度与背景干扰。

2.3 跟踪与部署

检测输出经ByteTrack关联得到稳定track_id。模型导出为ONNX格式，推理端采用隔帧检测与轨迹外推策略（每2帧推理1次），输出锁帧30 FPS，在Windows平台通过ONNX Runtime与DirectML加速，在消费级AMD GPU上实现实时运行。该策略在保持跟踪连续性的同时，显著降低算力占用。

3 拥堵指数算法设计

3.1 特征定义与物理含义

在垂直俯视条件下，系统直接基于二维像素坐标分析，无需三维投影变换。该假设适用于飞行高度与俯角相对固定的UAV巡检任务：在约80

m高度、 -90° 垂直俯拍下，道路平面近似与图像平面平行，车辆位移在像素空间与沿道路方向的运行状态具有单调对应关系，因此可用像素/帧作为速度单位，用像素作为车距单位。

每帧从清洗后的轨迹中提取以下特征：（1）平均速度 V_{avg} ，即滑动窗口内各车位移的算术平均；（2）帧车距 D_t ，即按车队切分后的队内相邻间距中位数；（3）车距波动 σ_{dist} ，即3s滑动窗口内 D_t 序列的标准差；（4）静止占比 r_{stop} ，即位移低于 v_{static} 阈值的车辆比例。该思路与张溪等[1]利用运行指数反映路网状态、时柏营等[2]综合速度与流量信息构建拥堵指数的做法一致，但本文特征直接来自视频轨迹而非线圈或浮动车。

3.2 数据清洗

DataCleaner对Y0L0v12与ByteTrack输出进行三层处理：一是EMA坐标平滑，抑制检测框边缘抖动；二是死区过滤（默认2px），消除无人机悬停造成的伪位移；三是3s滑动窗口，维护每辆车的轨迹历史。对于短暂丢失的 $track_id$ ，窗口内以空位占位，避免错误关联。上述处理是后续速度及车距计算可靠性的前提。

3.3 车队切分

路口或多车道场景下，若将所有车辆按一维排序，跨队列大间距会被计入 D_t ，从而抬高 σ_{dist} ，导致走走停停误判。本文沿方差最大轴（通常为道路走向轴）对车辆排序，当相邻间距超过 $GapThr$

$$= k \cdot (L_i + L_{i+1}) / 2 \quad (k=1.5)$$

时切分为不同车队，仅在同一车队内部统计车距，其中 L_i 为检测框跨度，表征"车长"。该策略本质上是在像素空间进行Platoon识别，避免把"空车道间隙"当作"车队内稀疏"。

3.4 连续状态机与子分融合

根据 V_{avg} 与 σ_{dist} 划分交通场景：A畅通（ $V_{avg} \geq V_{high}$ ，默认8px/帧）；B有序等待（ $V_{avg} \leq v_{low}$ 且 $\sigma_{dist} \leq \sigma_{low}$ ）；C走走停停（ $V_{avg} \leq v_{low}$ 且 $\sigma_{dist} \geq \sigma_{high}$ ）；D过渡或慢速跟车（中间速度区间）；E路口等灯（ $0.2 < r_{stop} < 0.8$ 且动组均速仍较高、静组波动小）。

速度子分 S_{speed} 与波动子分 S_{fluct} 分别映射至0—100，再加权合成原始分： $C_{raw} = \alpha \cdot S_{speed} + \beta \cdot S_{fluct}$ ， $\alpha=0.67$ ， $\beta=0.33$ ，体现"速度主导、波动辅助"的设计。B/C区间采用连续映射： $\sigma_{dist} \leq \sigma_{low}$ 时波动子分约20； σ_{low} 至 σ_{high} 线性升至80；超过 σ_{high} 后继续升至100。该设计借鉴朱云等[3]模糊分级思想，但采用可解释的分段连续函数，避免 σ 在阈值处20→80的跳崖。

路口场景下，若静止车辆共存且部分车道仍在放行，系统识别为E类并对Craw封顶约20—30分，避免“看起来在走却被判重度拥堵”。

3.5 抗抖动机制

UAV检测框抖动、车辆数瞬时变化易导致指数跳变，本文引入五项机制：（1）Hold挂起： $n < 2$ 时不清空历史队列，指数按6分/秒衰减，超过0.5 s无车才归零；（2）Dist Warmup：从无车/少车恢复后连续5帧才向Dt窗口追加，防止0与大值突变抬高 σ_{dist} ；（3）时间基EMA： $\tau = 1.5$ s， $\alpha_t = 1 - \exp(-dt/\tau)$ ，与FPS解耦；（4）速率限幅：单帧变化不超过8分/秒；（5）Episode Reset：长时间空窗后清空陈旧Dt样本。最终判定： $C_t < 60$ 为畅通， $C_t \geq 60$ 为拥堵。

4 系统实现

系统基于Python实现，核心模块包括traffic_analysis包（DataCleaner、CongestionCalculator）、analyzer.py视频分析引擎与app.py Web服务。

视频引擎采用生产者—消费者多线程模型：生产者线程读取视频、执行YOLOv12检测或轨迹预测、调用拥堵算法；消费者线程将标注帧编码为JPEG并更新最新指标，经WebSocket广播至前端。帧队列深度为2，避免积压造成延迟增大。

Web前端提供模型与视频选择、参数滑块热更新、开始/停止/暂停/重置控制、逐帧调试模式（Step

Mode）及全量指标CSV导出（25列，含raw分、Dt、WinMean、车队数、GapThr等中间量）。图表采用90 s滚动时间窗口，纵轴0—75压缩显示，橙色虚线标注60分阈值。

与潘旭[5]提出的“感知—平台—应用”四层体系相比，本文平台更轻量，但保留了实时可视化、参数热更新与逐帧调试等工程化能力，便于算法验证与成果演示。端口冲突时系统自动回退至8001等可用端口，提升部署友好性。

5 实验与分析

5.1 实验环境

实验平台为Windows PC，GPU采用AMD显卡并通过DirectML加速；软件环境为Python 3.9、Ultralytics YOLOv12、ONNX Runtime-DirectML、OpenCV与FastAPI。测试视频为自采集UAV俯视交通视频，涵盖路段直行、红灯排队、走走停停及路口混行等场景。算法时基统一锁定30 FPS，滑动窗口3 s，与Web图表90 s滚动窗口配合，便于观察拥堵指数的中短期变化。

5.2 典型场景分析

（1）自由流场景：车辆间距较大、运行速度较高， V_{avg} 高于 V_{high} ， σ_{dist} 维持低位， C_t 稳

定低于60，系统判定为畅通（A类）。

（2）红灯有序等待场景：车辆整体低速或静止，队列车距较为均匀， σ_{dist} 低于 σ_{low} ， Ct 维持在20—40区间，对应B类有序等待，不会出现0与100的剧烈跳变。

（3）走走停停场景：车辆频繁加减速，窗口内 Dt 呈双峰或大幅波动， σ_{dist} 高于 σ_{high} ， Ct 达到60以上，对应C类严重拥堵。

（4）路口等灯场景：部分车道静止、部分车道仍有过车， r_{stop} 介于0.2—0.8，系统识别为E类并对 Ct 封顶约30，避免与C类混淆。

5.3 工程验证与调试能力

系统提供逐帧调试模式（Step

Mode）与25列CSV导出，包含raw分、 Dt 、WinMean、WinLen、Platoons、GapThr、hold秒数等中间量，便于复现“指数为何升高/降低”的分析过程。该能力在算法开发阶段用于定位：检测抖动、空帧恢复、路口误切分等问题，具有较高研发价值。

5.4 实时性能

隔帧推理配合30

FPS锁帧后，系统可在消费级GPU上稳定运行。Web端视频流仅叠加检测框、无文字HUD遮挡，指标集中于页面侧栏与图表区。WebSocket实时推送延迟低于视频帧间隔，满足演示与监测需求。

5.5 讨论

对比时间序列预测类方法[1][2]，本文侧重实时研判而非未来时段预测；对比CNN端到端识别[6]，本文采用可解释特征与规则状态机，便于调试和机理分析；与广州等城市治理研究[7]的宏观政策视角互补，本文提供微观视频级的技术实现路径。后续可补充mAP、MOTA等定量指标，并与浮动车或路侧数据对比验证。

6 结 论

本文面向无人机垂直俯视交通视频，完成了从数据标注、模型训练、拥堵算法到Web系统的完整实现。主要结论如下：

（1）VisDrone与自标注微调的两阶段训练可有效提升俯视车辆检测适应性，X-AnyLabeling标注流程便于项目场景数据快速迭代。

（2）车队切分、连续状态机与Hold/Warmup等抗抖动机制能缓解路口误判和指数跳变问题， $Ct < 60$ 与 $Ct \geq 60$ 的判定规则简洁明确，便于工程应用。

（3）所构建的实时分析平台具备工程可用性，可为UAV交通监测、应急勘查及算法演示提供参考。

后续工作将补充定量检测与跟踪评价指标,开展更多场景对比实验,并探索与宏观拥堵指数[1][2]及多源大数据平台[4][5]的融合预测。

参考文献

- [1] 张溪, 温慧敏, 穆毅, 等. 基于时间序列法的城市道路交通拥堵研判与应用实例[J]. 智能交通与安全, 2012(8): 421-424.
- [2] 时柏营, 杜文鑫, 梁成, 等. 基于SSA-SVM的短时交通拥堵指数预测[J]. 黑龙江交通科技, 2025(10): 120-124.
- [3] 朱云, 王建宇, 杨影, 等. 城市道路交通拥堵的模糊神经网络评析[J]. 北京工业大学学报, 2018, 38(5): 487-492.
- [4] 盖健, 孙明铭, 李雪, 等. 大数据环境下道路拥堵智能疏导技术分析[J]. 数字经济, 2025(22): 94-95.
- [5] 潘旭. 大数据分析驱动的城市道路交通拥堵演化机理与疏导模型研究[C]//2025年第八届工程领域数字化转型与新质生产力发展研究学术交流会论文集, 2025: 105-107.
- [6] 罗运鹏, 李艺. 基于卷积神经网络的道路拥堵识别模型分析[J]. 战略连线, 2025(18): 174-176.
- [7] 黄晓虹, 崔昂. 广州城市道路状况演变分析及拥堵治理对策研究[J]. 交通与港航, 2025(5): 55-61.